OBJECT ORIENTED PROGRAMMING
BS(CS) MIDTERM EXAMINATION FALL 2024

Date:	Time: 90 Minutes	Max Marks: 50
Name:	Reg. #:	Section:

CLO #	Course Learning Outcome (CLO)	Taxonomy Level	PLO	Ques #	Marks
1	Describe key concepts of object-oriented paradigm	C2 (Understand)	2	1,2	15+15
2	Interpret real world problems in terms of object-oriented design	C3 (Apply)	3	3	20
3	Apply object-oriented programming principles to implement small and medium problems	C3 (Apply)	4	-	-

1. Write a program to simulate a physics experiment that tracks particles moving in 2D space, incorporating object-oriented programming principles. Each particle is characterized by its `position`, `velocity`, and `mass`, and should be represented using a class named `Particle`. Implement constructors for initializing particles, including a default constructor, a parameterized constructor with default argument for `velocity`, and an overloaded constructor that initializes only the `mass`. Use dynamic memory allocation to manage the `position` of particles and ensure proper resource management by implementing a copy constructor for deep copying. Include a method `display()` to print the particle's attributes. Add a method `updatePosition(double time)` to update the particle's position based on its velocity and the time elapsed ($newPosition = oldPosition + velocity \times time$). The program should support creating and manipulating particle objects, updating their positions based on velocity over time, and displaying their attributes. Demonstrate the independence of objects created using deep copying by modifying one copy without affecting the original. At the end of the program, ensure there are no memory leaks or dangling pointers.

```
int main() {

// Create a particle "p1" using the default constructor
// Display the attributes of "p1"

// Create a particle "p2" using the parameterized constructor
// Display the attributes of "p2"

// Create a particle "p3" using the overloaded constructor with mass
// Display the attributes of "p3"

// Create a deep copy "p4" of an existing particle "p2"
// Display the attributes of "p4"

// Modify the copied particle "p4"
// Display the attributes of "p4"

// Display the attributes of "p2" to show the independence of objects

return 0;
}
```

Sample Output:

```
Default particle:
Position: (0, 0), Velocity: (0, 0),
Mass: 1

Parameterized particle:
Position: (2, 3), Velocity: (1, 1.5),
Mass: 5

Overloaded particle (mass only):
Position: (0, 0), Velocity: (0, 0),
Mass: 10

Copy of parameterized particle (p4):
Position: (2, 3), Velocity: (1, 1.5),
Mass: 5

After updating p4's position:
Position: (4, 6), Velocity: (1, 1.5),
Mass: 5

Original particle (p2) remains unchanged:
Position: (2, 3), Velocity: (1, 1.5),
Mass: 5
```

2. Demonstrate a class `Employee` with attributes for an employee's ID, name, and monthly salary, along with data members to track the total number of employees and the total salary expenditure. The employee's ID is assigned during object creation and cannot be modified. The constructor should initialize the employee's attributes and related data members accordingly. Member functions should include `getEmployeeCount()` to return the total number of employees and `getTotalSalary()` to return the total salary expenditure. Additionally, implement a function `writeEmployeeDetails()` that takes

an **Employee** object by reference and writes the employee's details to a file "output.txt" instead of displaying them on the console. The program should demonstrate creating multiple **Employee** objects, writing their details to the file, and showing the total number of employees and the total salary expenditure by writing this summary information to the file as well, making it practical for real-world applications in employee management.

Sample output.txt:

```
Employee Details:
ID: 1, Name: Vasim, Salary: 100000
ID: 2, Name: Vaqar, Salary:  150000
ID: 3, Name: Aqib, Salary: 50000

Total Employees: 3
Total Salary Expenditure: 300000
```

3. Implement an inventory management system for a retail store, utilizing operator overloading to manage operations on inventory items. Each item in the inventory is represented by a class **InventoryItem** with attributes such as **itemID**, **itemName**, **price**, and **quantity**. The program should overload various operators: arithmetic operators to perform operations like adding or reducing stock quantities; increment and decrement operators to adjust the quantity of items by one unit; relational operators to compare items based on their price; insertion operator for the output of item details. The program should demonstrate creating multiple **InventoryItem** objects, adjusting item quantities, comparing items, and displaying item details using the overloaded operators.

```
int main() {

    InventoryItem item1(101, "Laptop", 1200.5, 10);
    InventoryItem item2(102, "Mouse", 25.75, 50);

    cout << "Initial Inventory:\n";
    cout << item1 << endl;
    cout << item2 << endl;

    item1 = item1 + 5;  // Add 5 to item1's quantity
    item2 = item2 - 10; // Subtract 10 from item2's quantity

    cout << "\nUpdated Inventory after adding/subtracting stock:\n";
    cout << item1 << endl;
    cout << item2 << endl;

    ++item1;  // Increment quantity of item1
    --item2;  // Decrement quantity of item2

    cout << "\nInventory after increment/decrement:\n";
    cout << item1 << endl;
    cout << item2 << endl;

    if (item1 > item2) {
        cout << "\n" << item1 << " is more expensive than " << item2 << endl;
    } else {
        cout << "\n" << item2 << " is more expensive than " << item1 << endl;
    }

return 0;
}
```

Sample Output:

```
Initial Inventory:
Item ID: 101, Name: Laptop, Price: $1200.5, Quantity: 10
Item ID: 102, Name: Mouse, Price: $25.75, Quantity: 50

Updated Inventory after adding/subtracting stock:
Item ID: 101, Name: Laptop, Price: $1200.5, Quantity: 15
Item ID: 102, Name: Mouse, Price: $25.75, Quantity: 40

Inventory after increment/decrement:
Item ID: 101, Name: Laptop, Price: $1200.5, Quantity: 16
Item ID: 102, Name: Mouse, Price: $25.75, Quantity: 39

Laptop is more expensive than Mouse
```